SQL 注入漏洞与防护

一、数据库与查询条件基础说明

1. 页面中的输入为什么会与数据库查询有关

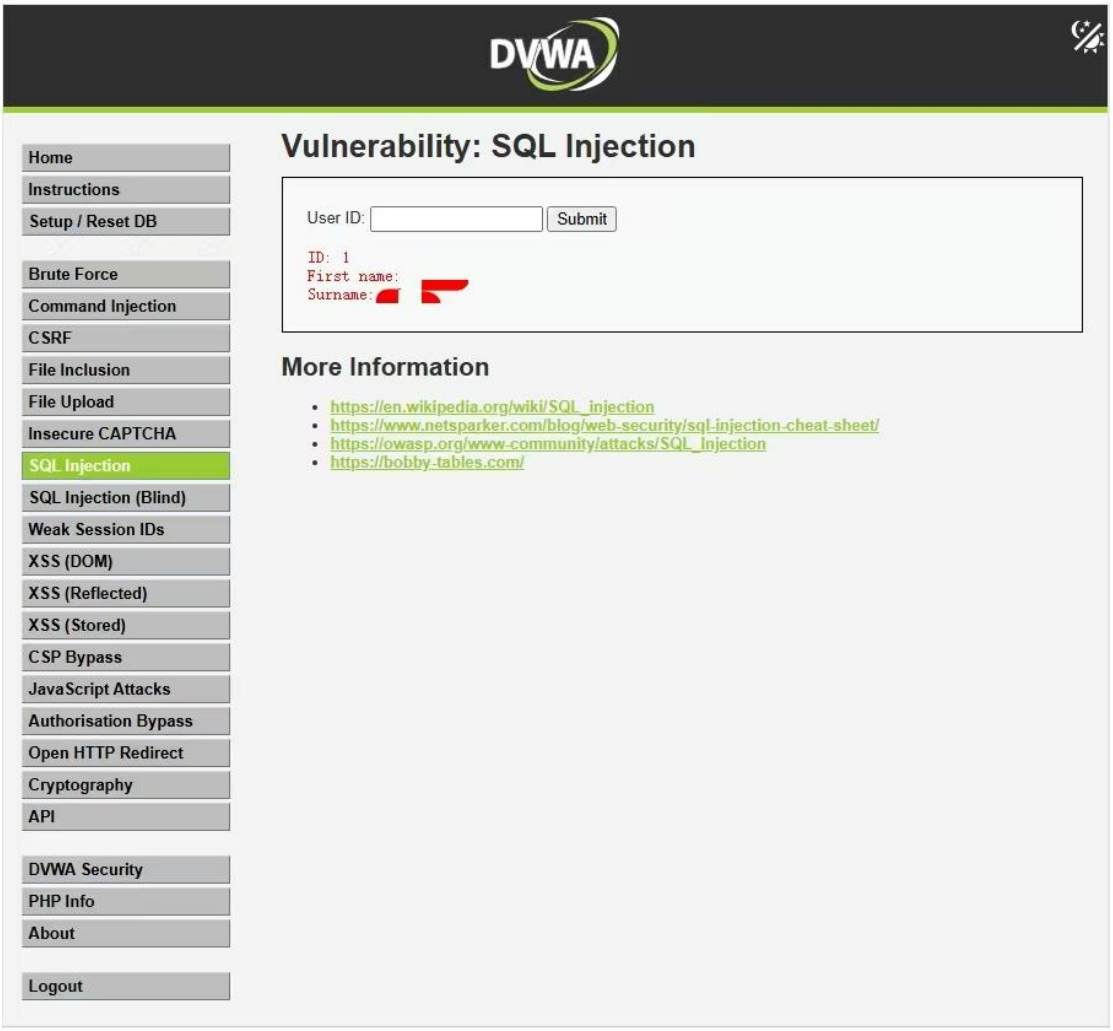
页面输入数据直接进入数据库中。可以从这几个方面来看，当输入不同时，返回结果不同；

当输入为空时，返回值为无结果或错误，以及登录必须对比已有数据，最后查询功能本质依赖数据。

换句话说如果没有数据库，登录和查询根本无法实现。最核心的一点就是输入不会直接变成页面内容，而是先参与查询，再影响结果。

2. 一个最小业务链路是什么

用户输入 → 查询条件 → 数据库执行 → 返回结果



举例：ID 查询

输入：ID=1

查询条件：ID=1

数据库执行

返回结果：Frist name：

Surname：

3. 用户输入是如何进入查询条件的

用户输入→前端处理→服务器接收并处理→进入数据库查询

举例：输入用户名登录
第一步：用户输入
用户输入：**username**
Password

第二步：前端处理
浏览器：收集输入、组装请求（**HTTP**）

注意：浏览器和服务器中间存在信任边界。 浏览器的数据一旦进入服务器，这些数据就变成需要“验证的输入”。换句话说服务器不能信任任何用户输入。

第三步：服务器接收并处理
服务器：接收输入、简单查询

第四步：进入数据库查询
数据库：核对用户、密码

到现在用户输入，已经进入数据库查询条件了。用户输入已经变成查询条件的一部分。用户输入由原始数据变为查询条件，就是从用户输入进入数据库查询开始。

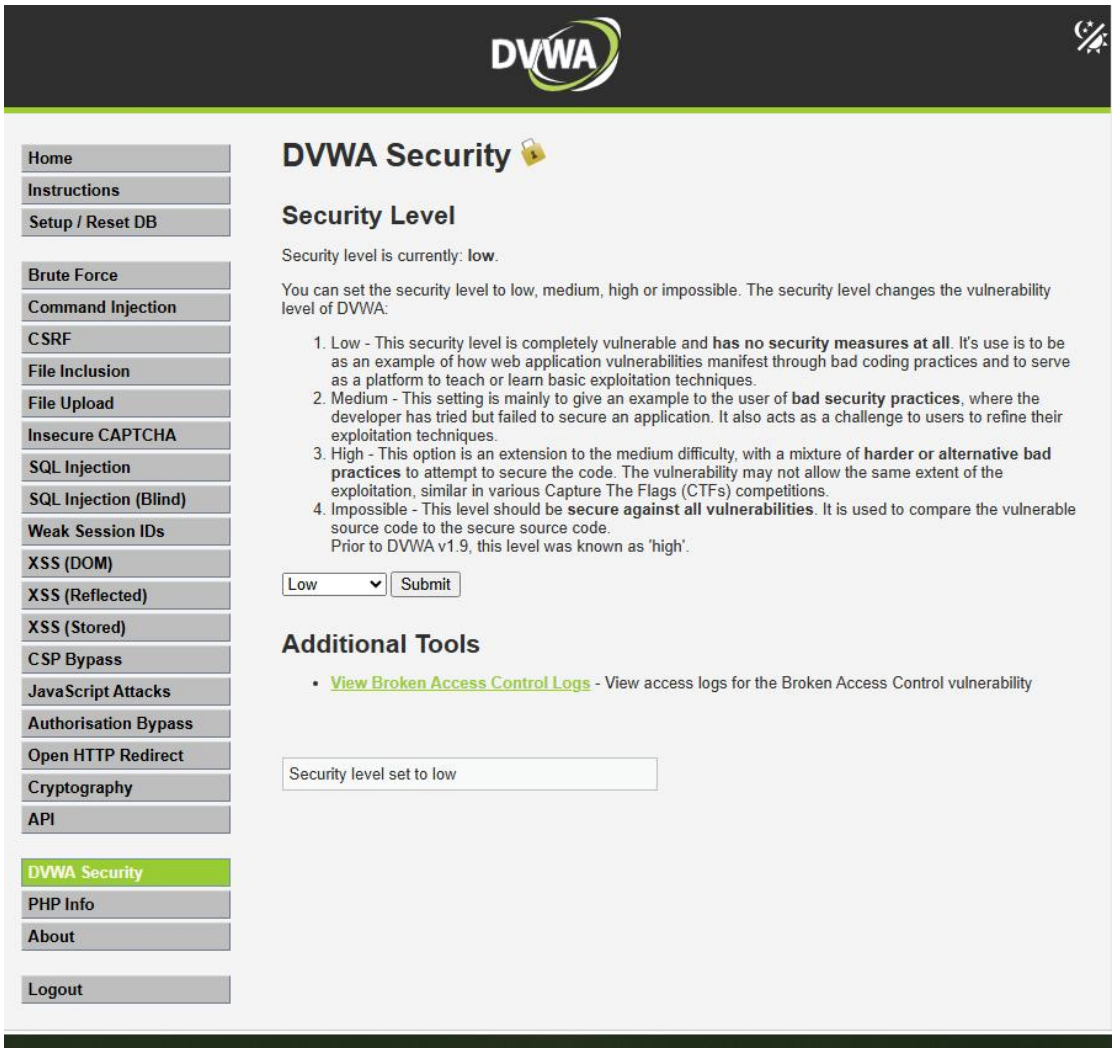
4. 为什么这为 SQL 注入创造了可能性

当用户输入的不是正常的 **username** 或 **id**，这些输入有可能被直接用于数据库查询或没有正确被处理。可能间接导致输入影响查询逻辑。

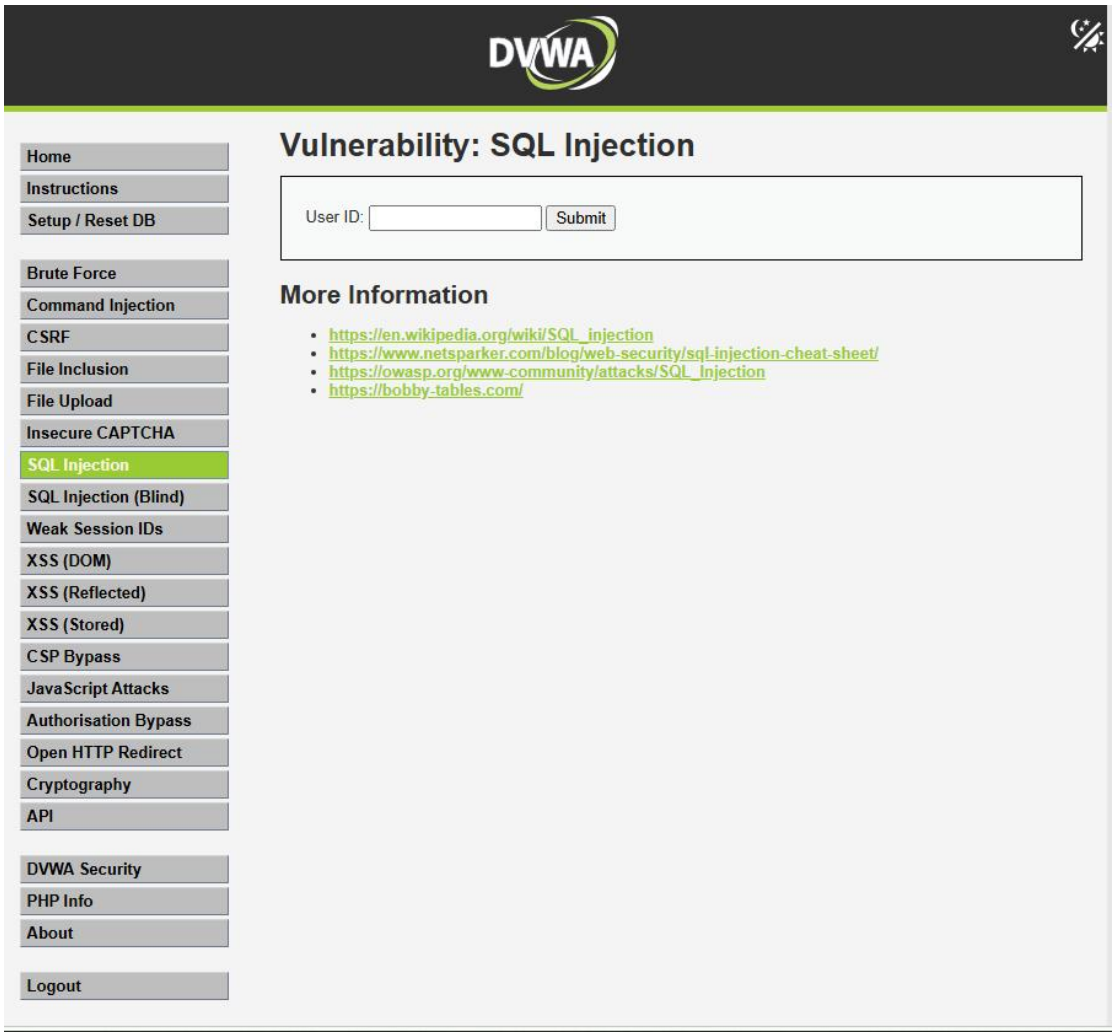
二、DVWA SQL Injection Low 现象记录

1. 实验界面或实验目标

1.将 **security level** 切换到 **low** 级别

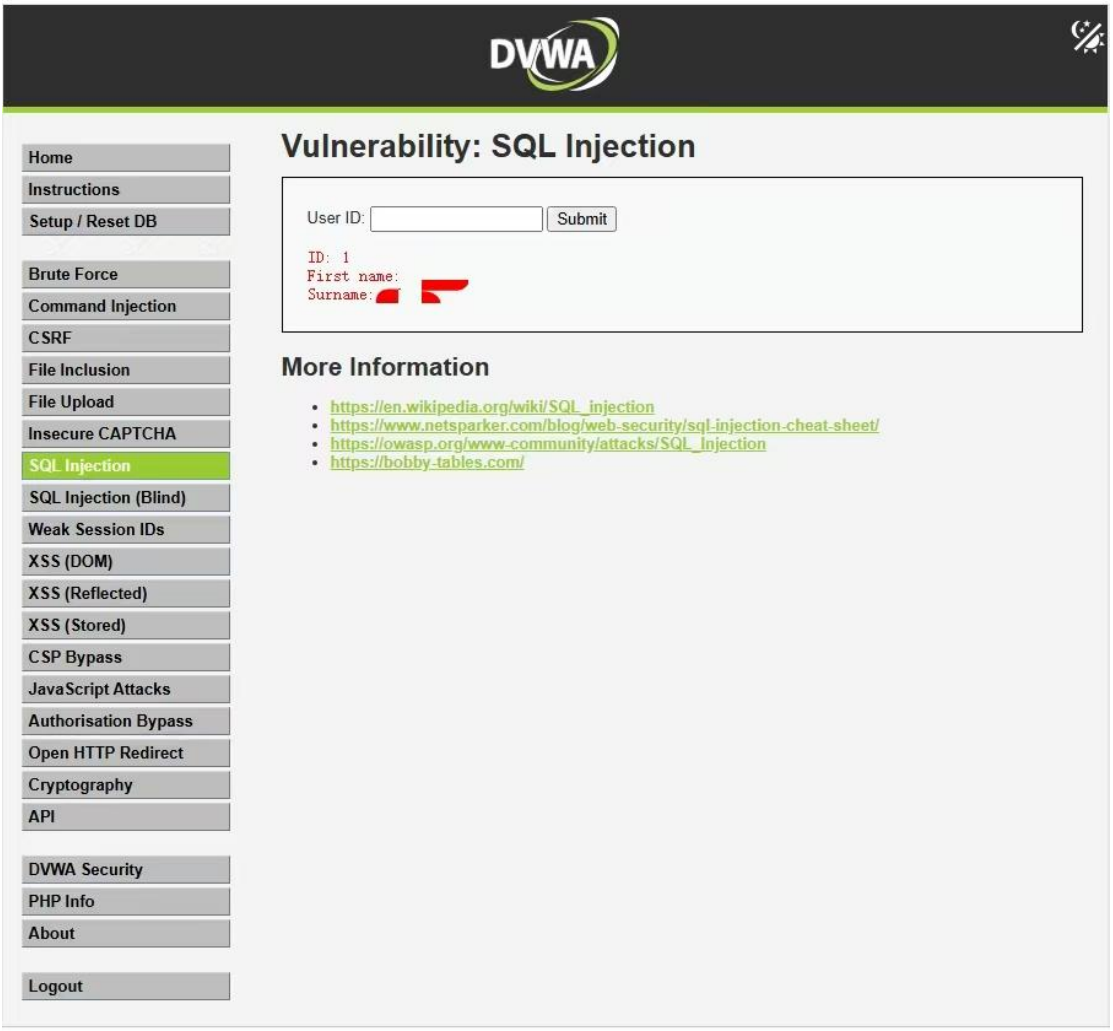

DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
API
DVWA Security
PHP Info
About
Logout
DVWA Security
Security Level
Security level is currently: low.
You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:
1. Low - This security level is completely vulnerable and has no security measures at all. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
2. Medium - This setting is mainly to give an example to the user of bad security practices, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
3. High - This option is an extension to the medium difficulty, with a mixture of harder or alternative bad practices to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
4. Impossible - This level should be secure against all vulnerabilities. It is used to compare the vulnerable source code to the secure source code.
Prior to DVWA v1.9, this level was known as 'high'.
Low
Submit
Additional Tools
View Broken Access Control Logs - View access logs for the Broken Access Control vulnerability
Security level set to low

2.实验界面

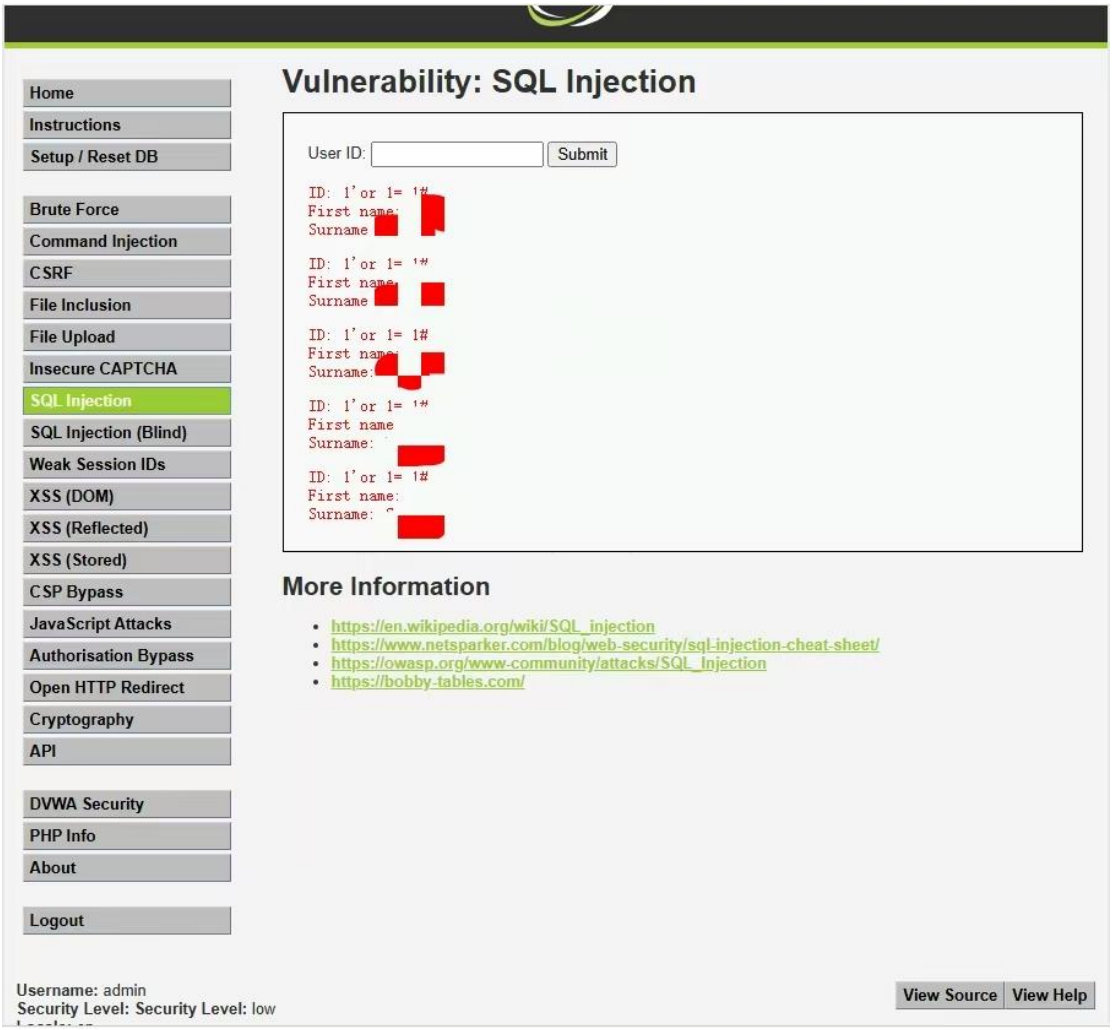


2. 普通输入与异常输入的对比

1. 普通输入


DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
API
DVWA Security
PHP Info
About
Logout
Vulnerability: SQL Injection
User ID:
Submit
ID: 1
First name:
Surname:
More Information
https://en.wikipedia.org/wiki/SQL_injection
https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/
https://owasp.org/www-community/attacks/SQL_Injection
https://bobby-tables.com/

2.异常输入



对比两种输入

项目	正常输入	异常输入
返回条数	1 条	多条
查询范围	单一 ID	多个 ID
结果	固定	异常变化

3. 返回差异

正常输入：输入 1

结果：返回用户 1

说明：输入只决定查询哪个用户

异常输入：输入 1' or 1=1#

结果：返回用户 1、2、3、4、5

说明：输入影响查询结果

根据两者返回差异得出，查询条件已经被改变

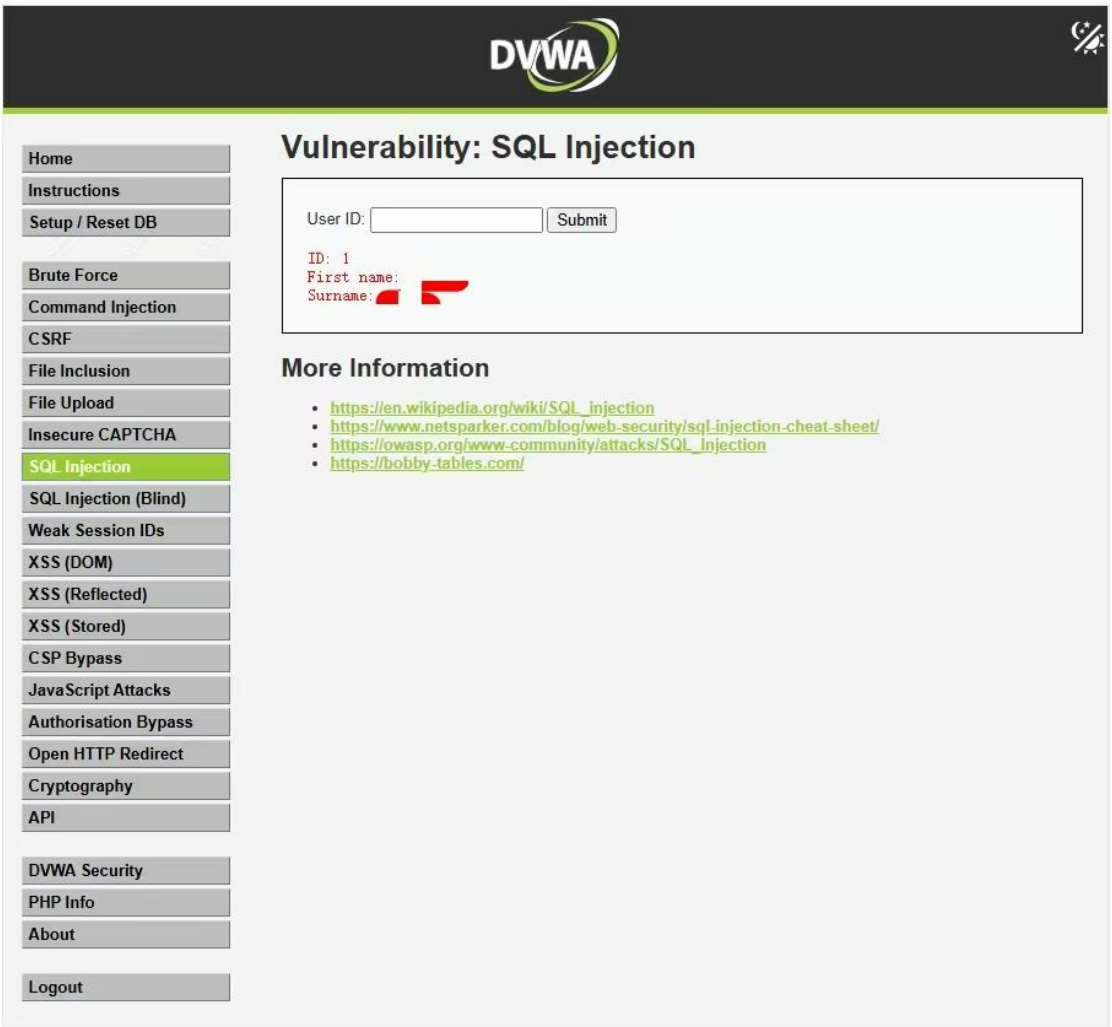
4. 为什么不像正常查询变化

因为输入不一样。输入已经不只是数据了，影响了查询逻辑。

三、机制分析与差异比较

1. 输入如何进入查询结构

用户输入→应用接收→拼接到查询条件→语句结构改变→数据库解析语句→数据库执行→返回结果



举例：输入 ID

第一步：用户输入 Use id：1

第二步：应用接收这个输入

第三步：应用直接把输入拼到 SQL 语句中

第四步：数据库收到“完整语句”并解析

第五步：执行解析结果

第六步：返回结果给界面

2. 为什么根因是“结构被污染”而不是“字符本身危险”

数据库不会判断“输入是不是恶意的”。数据库只是按照语句执行。关键在于输入变了，导致查询结果变了，最后执行结果变了。

3. Low 与 Medium 的差异在哪里

本质差别在于 low 是用户任意输入，medium 控制了输入方式，只能选择固定项。Low 直接拼接用户输入，medium 有一个基础的过滤。Low 在漏洞方面属于无防护状态，medium 有基础拦截但存在绕过的可能。

4. 普通 SQL 注入与 Blind SQL Injection 的差异在哪里

从五个方面来看，第一点页面回显，普通 SQL 注入是直接显示数据库错误或查询结果，而盲注页面不显示错误信息或着数据内容；第二点数据获取方式，普通 SQL 注入通过 UNION 联合查询直接获取数据，而盲注通过逻辑判断逐位推断数据；第三点响应特征，普通 SQL 注入响应错误信息、数据表内容直接可见，而盲注相应的仅页面内容差异或响应时间差异；第四点攻击效率，普通 SQL 注入高效，可批量提取数据，而盲注低效，需大量请求逐位猜测；第五点利用条件，普通 SQL 注入应用显示详细错误信息，而盲注应用隐藏错误但存在逻辑判断点

5. 为什么盲注与普通注入根因相同，但证据形式不同

他们的根因相同（本质都是 SQL 注入）：1.应用程序都将用户输入直接拼接到 SQL 语句中 2.都未使用预编译语句处理用户输入 3.数据库默认执行应用传入的 SQL 语句

证据形式不同的原因：

1. 错误处理机制差异

普通注入：应用开启错误显示，数据库错误直接输出到页面

盲注：应用关闭错误显示，但 SQL 逻辑仍被执行

2. 数据回显策略差异

普通注入：查询结果直接渲染到 HTML 页面

盲注：查询结果不输出，仅通过业务逻辑影响页面

3. 防御措施程度差异

普通注入场景：几乎无防护，原始漏洞状态

盲注场景：已采取部分防护（如关闭错误显示、过滤敏感词），但未从根本上解决动态拼接问题

四、SQL 注入防护清单与解释

1. 参数化查询

解决问题：

1. 从根本上解决动态 SQL 拼接漏洞，将 SQL 代码与数据分离
2. 防止攻击者通过输入恶意 SQL 代码篡改原始查询逻辑
3. 确保用户输入始终被当作数据处理，而非 SQL 命令执行

控制层级：根因层

1. 直接消除 SQL 注入的根本原因——动态 SQL 拼接
2. 从代码层面切断攻击路径

不可替代：

1. 不能替代输入校验
2. 不能替代业务逻辑验证
3. 不能替代错误处理机制

2. 输入校验

解决问题：

1. 在数据进入系统前进行过滤和验证
2. 阻止恶意字符、SQL 关键字、异常长度数据进入应用
3. 确保输入符合预期的数据类型、格式、范围

控制层级：暴露层面

1. 确保输入符合预期的数据类型、格式、范围
2. 作为第一道防线，降低恶意数据到达数据库的可能性

不可替代：

1. 不能替代参数化查询
2. 不能假设"校验过就安全"，必须纵深防御
3. 不能替代数据库层的权限控制

3. 错误处理

解决问题：

1. 防止数据库错误信息直接暴露给用户
2. 避免攻击者通过错误回显获取数据库结构、表名、字段名等敏感信息
3. 阻止基于错误的注入攻击

控制层级：暴露层面

1. 隐藏系统细节，增加攻击者信息收集难度
2. 将详细错误记录到日志，向用户展示通用错误消息

不能替代：

1. 不能替代参数化查询
2. 不能替代输入校验
3. 只是"安全通过隐匿"，属于辅助措施而非根本解决方案

4. 最小权限

解决问题：

1. 限制数据库账户的操作权限，仅授予完成业务所需的最小权限
2. 即使发生 SQL 注入，攻击者也无法执行高危操作
3. 降低注入攻击成功后的危害范围

控制层级：后果范围层

1. 属于损害控制措施，限制攻击成功后的影响
2. 防止权限提升和数据大规模泄露

不能替代：

1. 不能替代参数化查询
2. 不能替代输入校验和错误处理
3. 只是减轻后果，不能阻止攻击发生

5.可补充其他合理措施

Web 应用防火墙

解决问题：

1. 实时检测和拦截 SQL 注入攻击流量
2. 基于规则或机器学习识别恶意 SQL 语句模式
3.提供虚拟补丁，在应用修复前提供临时保护

控制层级：暴露层+后果范围层

1. 在网络层拦截攻击请求
2.作为应用层防护的补充

不能替代：

1. 不能替代代码层的参数化查询
2. 规则可能被绕过（编码、分块传输等）
3.属于外围防护，不能修复应用本身漏洞

五、SQL 注入因果链

①输入点

↓

用户输入：**username=‘admin’**

↓

构造恶意输入：

Username=‘admin’ or ‘ 1 ’ = ’1’

↓

②查询构造

↓

应用程序动态拼接 **SQL**：

Sql=“select * from users

Where username=' ” +userInput + “ ’ ”

↓

最终 **SQL** 语句：

select * from users

Where username=‘admin’ or ‘1’ = ‘1’

↓

③数据库解析/执行

↓

数据库解析 SQL 语句：
识别为 where 条件为恒真式
执行查询返回所有用户数据

↓

④结果差异

↓

攻击者观察到的证据：
页面返回所有用户数据
登录成功绕过
数据库报错信息暴露表结构

↓

⑤根因判断

↓

根本原因分析：

1. 动态 SQL 拼接漏洞
2. 未对用户输入进行验证
3. 数据库信任应用程序输入
4. 信息错误直接暴露

六、学生自评表

1.我本章最清楚的一点是什么？

SQL 注入的根本原因与参数化查询的防护原理

通过本章学习，我最清楚的是 SQL 注入的核心根因——动态 SQL 拼接漏洞。我理解了当应用程序直接将用户输入拼接到 SQL 语句中时，攻击者可以通过构造特殊输入（如 ' OR '1'='1）来篡改原始 SQL 语句的逻辑结构。

同时，我也清楚了**参数化查询（预编译语句）**为什么能从根本上解决这个问题：它将 SQL 代码与数据分离，数据库在编译阶段就已经确定了 SQL 语句的结构，后续传入的参数无论包含什么内容，都只会被当作数据处理，而不会被解析为 SQL 命令。这种"代码与数据分离"的机制从根因层面切断了攻击路径

2.我最容易混淆的一点是什么？

SQL 注入的根本原因与参数化查询的防护原理

通过本章学习，我最清楚的是 SQL 注入的核心根因——动态 SQL 拼接漏洞。我理解了当应用程序直接将用户输入拼接到 SQL 语句中时，攻击者可以通过构造特殊输入（如 ' OR '1'='1）来篡改原始 SQL 语句的逻辑结构。

同时，我也清楚了**参数化查询（预编译语句）**为什么能从根本上解决这个问题：它将 SQL 代码与数据分离，数据库在编译阶段就已经确定了 SQL 语句的结构，后续传入的参数无论包含什么内容，都只会被当作数据处理，而不会被解析为 SQL 命令。这种"代码与数据分离"的机制从根因层面切断了攻击路径

3.我本次作业中哪一段最需要教师复核？

防护控制点的层级划分与"不能替代"的说明

我最需要教师复核的是任务 4 中各防护措施控制的层级判断，特别是：

输入校验我判断为"暴露面层"，但不确定是否也应该归类为"根因层"的辅助措施
错误处理我判断为"暴露面层"，但它是否也属于"后果范围层"的减轻措施
各措施"不能替代什么"的说明是否准确，例如我写"输入校验不能替代参数化查询"，这个逻辑是否正确

4.如果重做一次，我会优先改哪一部分？

SQL 注入因果链关系图的可视化表达
如果重做一次，我会优先改关系图可视化：

1.增加实际代码示例：在"查询构造"环节加入具体的不安全代码和安全代码对比
2.添加攻击流程图：用流程图展示攻击者从信息收集到数据提取的完整步骤
3.强化防护对应关系：在因果链的每个环节明确标注"如果没有该防护会怎样"和"有了该防护如何阻断"
4.补充真实案例：加入一个具体的 SQL 注入攻击案例（如某网站数据泄露事件），标注该案例中哪个防护环节缺失导致了攻击成功

另外，我会增加盲注的因果链分支，展示盲注与普通注入在"结果差异/行为证据"环节的不同表现。

七、AI 输出审核与人工复核记录

1. 本作业哪些内容使用过 AI 辅助

我使用 AI 辅助完成的内容包括：
任务 4：SQL 注入防护清单与解释
AI 帮助整理了 5 项防护措施（参数化查询、输入校验、错误处理、最小权限、WAF）的框架结构
AI 提供了"它解决什么问题、它控制哪一层、它不能替代什么"的三段式说明模板
AI 生成了防护措施层级总结表的部分内容
收口项 1：SQL 注入因果链关系图
AI 协助设计了因果链的 5 个环节（输入点→查询构造→数据库解析/执行→结果差异→根因判断）
AI 提供了 ASCII 艺术图的基本框架
AI 帮助梳理了 4 个防护控制点的对应关系
收口项 2：学生自评表
AI 提供了自评问题的思考角度和回答框架
AI 建议了"容易混淆的点"可能的方向（普通注入 vs 盲注）

2.哪些判断是你自己复核后保留的

经过我自己复核后保留的判断包括：
防护层级的划分
参数化查询=根因层：我复核了课件内容，确认"动态 SQL 拼接"是根因，而参数化查询直接消除了这个根因，因此判断正确
输入校验=暴露面层：我复核后认为，输入校验虽然能减少攻击面，但不能从根本上解决漏洞（可能被绕过），因此属于暴露面层而非根因层
错误处理=暴露面层：我确认错误处理只是"隐匿信息"，不能修复漏洞，属于辅助措施
最小权限=后果范围层：我复核后确认，最小权限只是"减轻后果"，不能阻止攻击发生
"不能替代"的逻辑
我复核确认：输入校验不能替代参数化查询，因为校验可能被编码绕过、大小写混合等方式突破
我复核确认：错误处理不能替代参数化查询，因为"隐藏错误≠修复漏洞"
因果链的完整性

我复核了课件中 SQL 注入的核心原理："动态 SQL 拼接漏洞"、"数据库信任应用程序输入"、"SQL 语法特性利用"
我确认因果链的 5 个环节能完整覆盖从攻击入口到根因判断的全过程

3. 哪些内容你发现 AI 输出不准确并进行了修正

我发现并修正的 AI 输出内容包括：
修正 1：WAF 的层级归属
AI 初始输出：将 WAF 归类为"根因层"
我的修正：改为"暴露面层+后果范围层"
修正理由：WAF 是在网络层拦截攻击流量，属于外围防护，不能修复应用本身的代码漏洞。它既在入口处拦截攻击（暴露面层），也能限制攻击成功后的影响（后果范围层），但绝不是根因层
修正 2：输入校验的作用范围
AI 初始输出：强调输入校验可以"完全阻止 SQL 注入"
我的修正：改为"减少攻击面，作为第一道防线"
修正理由：根据课件内容，输入校验可能被绕过（如编码绕过、双写绕过、大小写混合等），不能假设"校验过就安全"，必须纵深防御
修正 3：错误处理的定位
AI 初始输出：将错误处理描述为"主动防御措施"
我的修正：改为"安全通过隐匿，属于辅助措施而非根本解决方案"
修正理由：课件强调"隐藏错误不等于修复漏洞"，错误处理只是增加攻击者信息收集难度，不能阻止攻击发生
修正 4：因果链中的"结果差异"描述
AI 初始输出：只提到"页面返回数据"
我的修正：补充了"数据库报错信息暴露表结构"、"响应时间差异"（针对盲注）
修正理由：根据课件，SQL 注入的证据形式包括错误回显、数据回显、时间延迟等多种，不能只强调数据回显

4. 你最终如何确认结论和证据是一一对应的

我通过以下方法确认结论和证据的一一对应关系：
（1）溯源课件原文
对于每个防护措施的层级判断，我都回溯到课件中的对应内容
例如：课件明确指出"动态 SQL 拼接漏洞"是核心原理，因此我确认"参数化查询"直接解决这个根因
课件提到"数据库默认认为应用程序传入的 SQL 语句是可信的"，这验证了"数据库信任应用程序输入"的根因
（2）逻辑链条验证
我建立了"问题→原因→措施→效果"的完整逻辑链：

问题：SQL 注入攻击成功→原因：动态 SQL 拼接+缺乏输入验证→措施：参数化查询+输入校验→效果：攻击者无法篡改 SQL 逻辑

每个环节都能前后呼应，不存在逻辑断裂